LAB: Stepper Motor

LAB: Stepper Motor

Date: 2026-10-15

Author/Partner: Suhyun Son

Demo Video: demo video link

Problem : Stepper Motor Control with FSM

Problem Description

Implement full-stepping and half-stepping for a stepper motor using an FSM (Finite State Machine), and determine the number of steps required for one full revolution in each mode, as well as the operation at 1 RPM in both forward and reverse directions. Additionally, measure the motor's maximum rotational speed under no-load conditions, and implement an emergency stop feature that halts the motor when the BT2 button is pressed, utilizing an external interrupt.

State Table for Full-Stepping

Full-stepping sequence

Phase	Port_Pin	1	2	3	4
A	PB_10	H	L	L	H
B	PB_4	H	H	L	L
A'	PB_5	L	H	H	L
B'	PB_3	L	L	H	H

State	Next State (DIR=0)	Next State (DIR=1)	Output (A B A' B')
S0	S1	S3	HHLL
S1	S2	S0	LHHL
S2	S3	S1	LLHH
S3	S0	S2	HLLH

State Table for Half-Stepping

Half-stepping sequence

Phase	Port_Pin	1	2	3	4	5	6	7	8
A	PB_10	H	H	L	L	L	L	L	H

Phase	Port_Pin	1	2	3	4	5	6	7	8
B	PB_4	L	L	L	H	H	L	L	L
A'	PB_5	L	H	H	H	L	L	L	L
B'	PB_3	L	L	L	L	L	H	H	H

State	Next State (DIR=0)	Next State (DIR=1)	Output (A B A' B')
S0	S1	S7	HLLL
S1	S2	S0	HLHL
S2	S3	S1	LLHL
S3	S4	S2	LHHL
S4	S5	S3	LHLL
S5	S6	S4	LHLH
S6	S7	S5	LLLH
S7	S0	S6	HLLH

Programming FSM

Function Definitions

```
//State number
typedef enum {
    S0, S1, S2, S3, S4, S5, S6, S7
} StateNum;


// Stepping sequence  - FSM
typedef struct {
  uint32_t next[2];
  uint8_t out[4];
} StateStepper_t;

// Initialization of Stepper Motor
void stepper_init(uint32_t mode, PinName_t *pinStepper);

// Converts Motor [rpm] to step delay in [msec]
void stepper_speed (uint32_t speedRPM);

// Run Stepper Motor for given steps and direction
void stepper_step(uint32_t steps, uint32_t direction);

// Stepper Motor Output for given state
void stepper_out(uint32_t state);

// Stop Stepper Motor
```

```
void stepper_stop(void);
```

```
static const StateStepper_t fsm_Full[4] = {
    {{S1,S3},{HIGH,HIGH,LOW,LOW}},
    {{S2,S0},{LOW,HIGH,HIGH,LOW}},
    {{S3,S1},{LOW,LOW,HIGH,HIGH}},
    {{S0,S2},{HIGH,LOW,LOW,HIGH}}
};


static const StateStepper_t fsm_Half[8] = {
    {{S1,S7},{1,0,0,0}},
    {{S2,S0},{1,1,0,0}},
    {{S3,S1},{0,1,0,0}},
    {{S4,S2},{0,1,0,1}},
    {{S5,S3},{0,0,0,1}},
    {{S6,S4},{0,0,1,1}},
    {{S7,S5},{0,0,1,0}},
    {{S0,S6},{1,0,1,0}}
};


// Initialization of Stepper Motor
void stepper_init(uint32_t mode, PinName_t *pinStepper){
    stepperMode = mode;
    emergencyStop = 0;

    if (mode == HALF)
        fsm = fsm_Half;
    else
        fsm = fsm_Full;

    state = S0;

    //  GPIO Digital Out Initiation
    // For A, B, AN, BN

    for (int i=0; i<4; i++){
        stepperPins[i] = pinStepper[i];
        GPIO_init(stepperPins[i], OUTPUT);
        GPIO_ospeed(stepperPins[i], FAST_SPEED);
    }
}


// Converts Motor [rpm] to step delay in [msec]
void stepper_speed (uint32_t speedRPM){      // rpm
```

```
    if (speedRPM == 0) return;

    uint32_t stepsPerRev = STEP_PER_REV;

    if (stepperMode == HALF)
        stepsPerRev = STEP_PER_REV * 2;

    _step_delay = 60*1000/(stepsPerRev*speedRPM);  //in [msec]

    if (_step_delay == 0)
        _step_delay = 1;

    // [usec] can be used for a higher accuracy
    // But needs delay_us()
}


// Run Stepper Motor for given steps and direction
void stepper_step(uint32_t steps, uint32_t direction){
    // Exit if stepper_init() not called
    if (fsm == 0 || emergencyStop) return;

    _steps = steps;

    // run for step size:
    for (uint32_t i = 0; i < _steps; i++){
        if (emergencyStop) break;

        // Update Present State
        state = fsm[state].next[direction];

        // Output of Present State
        stepper_out(state);

        // Delay for steppermotor speed
        delay_ms(_step_delay);
    }
}


// Stepper Motor Output for given state
void stepper_out(uint32_t state){

    // stepperPins[0]=A, stepperPins[1]=B, ...
    // out[0]=A, out[1]=B, ...

    for(int i =0; i<4;i++){
        GPIO_write(stepperPins[i], fsm[state].out[i]);
    }
}


// Stop Stepper Motor
void stepper_stop(void) {
```

```
    emergencyStop = 1;
    _steps = 0;

    // All pins DigitalOut '0'
    for(int i =0; i<4;i++){
        GPIO_write(stepperPins[i], LOW);
    }
}
```

Code

```
#include "stm32f411xe.h"
#include "ecSTM32F4v2_22400365.h"

// ( A, B,  AN,  BN)
PinName_t pinStepper[4] = {PB_10,PB_4,PB_5,PB_3};

// Emergency Stop
volatile uint8_t emergencyStop = 0;

// Motor setting
uint32_t mode = FULL;
uint32_t speedRPM = 1;

// Initialiization
void setup(void){
    RCC_PLL_init();                                 // System Clock = 84MHz
    SysTick_init();                                 // Systick init

    GPIO_init(BUTTON_PIN, INPUT);                   // GPIOC pin13 initialization
    EXTI_init(BUTTON_PIN, FALL,0);                  // External Interrupt Setting

    stepper_init(mode, pinStepper);                 // Stepper GPIO pin
initialization
    stepper_speed(speedRPM);                        // set stepper motor speed
}


int main(void) {
    // Initialiization --------------------------------------------------------
    setup();
    uint32_t rev=1;

    // Set steps according to stepping mode
    uint32_t steps=STEP_PER_REV*rev;

    if(mode == HALF){
        steps=STEP_PER_REV*2*rev;
    }
```

```
    // Inifinite Loop ------------------------------------------------------------
    while(1){

        // Emergency Stop
        if(emergencyStop){
            stepper_stop();
            while(1);
        }

        stepper_step(steps, DIR_CW);  // Direction : 0 or 1, Mode : FULL or HALF

        if(emergencyStop){
            stepper_stop();
            while(1);
        }

        delay_ms(2000);

        stepper_step(steps, DIR_CCW); // Direction : 0 or 1, Mode : FULL or HALF

        if(emergencyStop){
            stepper_stop();
            while(1);
        }

        delay_ms(2000);
    }
}

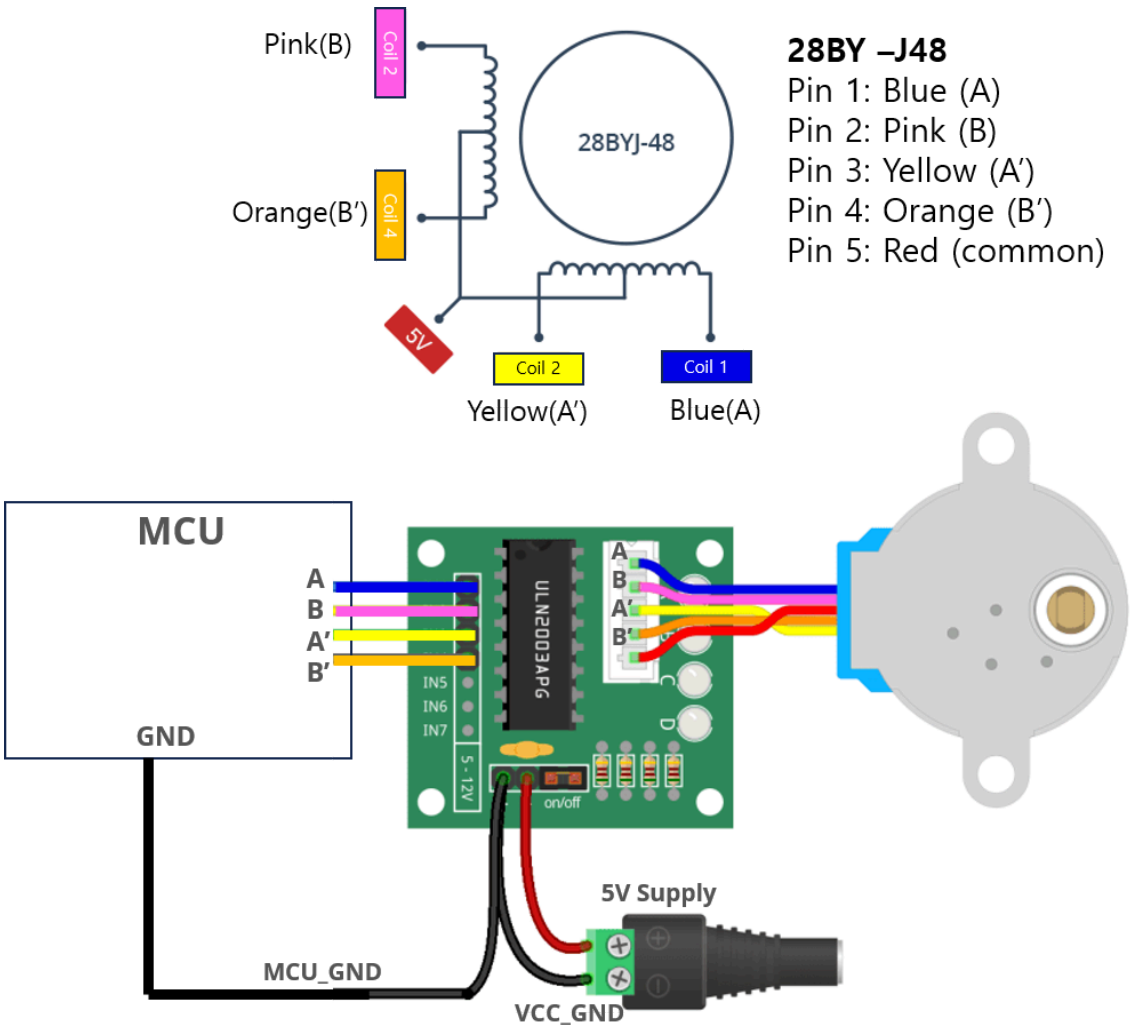

void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(BUTTON_PIN)) {
        emergencyStop = 1;
        stepper_stop();
        clear_pending_EXTI(BUTTON_PIN); // cleared by writing '1'
    }
}
```

Configuration

Function	Pins	Configuration
A, B, A', B'	PB10, PB4, PB5, PB3	DOUT, FAST

Connection Diagram

Read the specification sheet of the motor and the motor driver for wiring and min/max input voltage/current.



Discussion

1. Find out the trapezoid-shape velocity profile for a stepper motor. When is this profile necessary?

The trapezoidal velocity profile is a control method in which the motor accelerates to a constant speed, maintains that speed, and then decelerates. It is necessary to prevent the motor from missing steps or vibrating due to sudden changes in speed.

2. How would you change the code more efficiently for micro-stepping control? You don't have to code this but need to explain your strategy.

To implement micro-stepping, the control method is shifted from simple HIGH/LOW outputs to the precise regulation of current in each coil, based on sine and cosine waveforms. This can be achieved efficiently using PWM (Pulse Width Modulation) and a lookup table. However, since the current configuration—consisting of a ULN2003 driver and a 28BYJ-48 stepper motor—has limitations regarding precise current control, it is more appropriate to use a dedicated motor driver equipped with built-in current control capabilities for an actual micro-stepping implementation.

3. There are other types of Stepper Motor Drivers that are simple to use, such as you only give one pulse signal and direction, instead of giving 4 pulse signals. Such examples are A4988, DRV 8834, and TB6600 drivers. Compare these motor drivers with ULN2003 in terms of operating method.

With the ULN2003, the MCU must directly control the coil excitation sequence using four GPIO outputs. In contrast, the A4988, DRV8834, and TB6600 use a STEP signal to control the amount of rotation and a DIR

signal to control the direction of rotation, while their internal circuitry manages the coil current and excitation sequence. Consequently, the A4988, DRV8834, and TB6600 impose a lower control burden on the MCU compared to the ULN2003 and make it easier to implement micro-stepping.

Results

demo video link